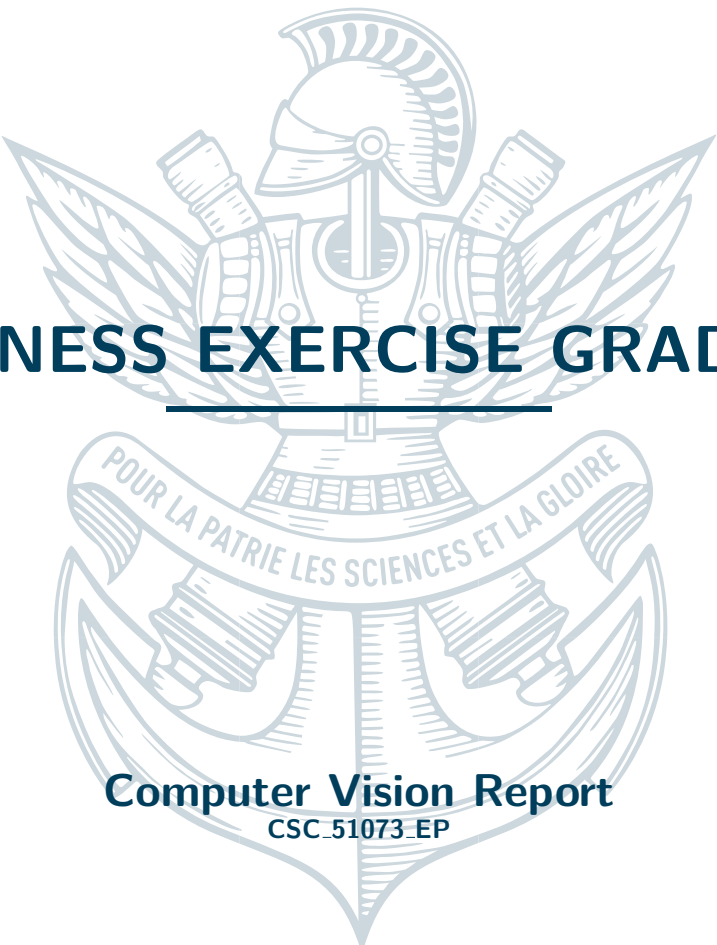




FITNESS EXERCISE GRADER

Computer Vision Report
CSC_51073_EP

September 13, 2026

P41 – Roman Lendormy, Arthur Buis
`roman.lendormy@polytechnique.edu`
`arthur.buis@polytechnique.edu`



1

INTRODUCTION

The rise of sports and digital coaching apps raises a key question: how can human movement be automatically analyzed from a simple video, without specialized equipment, while providing relevant feedback on the nature of the movement and its quality of execution? Being able to identify an exercise, count repetitions, and assess their correct execution is an important challenge for personal training, injury prevention, and rehabilitation. Our project addresses this issue and aims to design a complete computer vision pipeline for analyzing fitness exercise videos.

To do this, we chose to work with human position indicators using the MediaPipe library. Based on these anatomical landmarks, we built functions for data normalization, exercise classification, repetition detection, and finally movement quality assessment by comparison with a reference performance.

This report presents the different stages of this pipeline, the methodological choices that guided its development, as well as the difficulties encountered and the compromises adopted. The results obtained make it possible to assess the extent to which an approach based on pose estimation can be a viable solution for the automatic analysis of physical exercises from user videos.

2

REFERENCES AND AREAS OF FOCUS

We based our work on various scientific articles, which we will briefly introduce in this section and refer to throughout the report.

The paper that mainly guided our approach and the sub-phases of the project is an article entitled “Using Pose Estimation Algorithms to Build a Simple Gym Training Aid App” by Pawel Kapica, published in 2019 [1]. In this article, the author implemented the same kind of pipeline that we did, i.e., extraction of pose landmarks in the video, comparison with a reference execution of the exercise (by a professional), and scoring using synchronization of the two videos enabled by DTW and cosine similarity. However, we chose to add an exercise classifier to the pipeline in order to generalize and improve this approach. The presence of a machine learning model required us to have a training set: we first considered using Deepmind’s Kinetics-700 dataset [2], which includes 635,000 videos of 700 different human activities, but the number of categories identified as fitness exercises was very limited (only bench pressing and push-ups), so we opted for a more specialized dataset, even if it meant accepting lower quality. To do this, we used the Kaggle dataset “Gym Workout/Exercise Video” [3], which contains numerous videos that have already been processed, as well as less clean videos scraped from YouTube, allowing us to test the robustness of our program on videos that are more complicated to process. The first phase of our project therefore consisted of cleaning and exploiting this database of 980 videos describing 22 different classes of exercises.

In order to make the landmark pose data extracted from the videos usable by our classification

model, we had to carry out a fairly significant amount of pre-processing. The extraction was done using MediaPipe, and the poor angle of some videos in the dataset (legs or torso hidden by an object, for example) required us to clean it up by removing the poor videos. Then, in order to minimize variations in our training data to achieve a performing scoring method, we sought to implement a rotation- and translation-invariant version of our pose data. To do this, we drew inspiration from the article [4], which deals with human action recognition and proposed robust data pre-processing.

The article 3D Human Pose Classification using MediaPipe and PointNet [5] provided us with an initial methodological basis for tackling the task of exercise classification. It proposes directly exploiting MediaPipe’s 3D outputs in the form of a point cloud and processing them using the PointNet model [6], initially designed for the classification and segmentation of 3D point clouds. The PointNet model was introduced by Qi et al. in 2017. It accepts as input an unordered set of points in $\mathbb{R}^3$, without any assumptions about their spatial organization. This property is particularly well suited to the human pose data we are working with, which can naturally be viewed as a point cloud corresponding to the joints of the body. The PointNet architecture is based on two key principles. On the one hand, each point in the cloud is processed independently by a series of fully connected layers (shared MLPs), which allows local descriptors to be extracted for each point. On the other hand, a symmetric max-pooling operation is applied to all points in order to aggregate these descriptors into a global representation of the cloud.

In order to strengthen the model’s robustness to geometric transformations, PointNet also introduces “T-Net” modules, which are responsible for learning affine transformations that automatically align the point cloud in a coordinate system that is more favorable for classification. This invariance to rotations and translations made the model particularly interesting to us, as we were considering working with scenarios where the position of the camera or the orientation of the subject could vary. However, despite the theoretical relevance of this approach, we quickly observed its limitations in our context. The point clouds of the human body are too sparse, heavily noisy due to MediaPipe detection errors, and the movements of the different exercises are too similar. These factors prevented us from training the model in a relevant way, which ultimately led us to explore approaches with simpler models, MLP and CNN-1D. We also added temporal aggregation and classification at the video level rather than the frame level.

Finally, to apply the scoring approach proposed by [1], we first had to detect each exercise repetition in the video. To do this, we adopted a signal processing-based approach, considering the movement along a specific axis of a given body part in order to study these oscillations. We were particularly inspired by the approach introduced in the article Real-Time Posture Correction in Gym Exercises: A Computer Vision-Based Approach for Performance Analysis, Error Classification and Feedback [7], which proved to us that detecting extrema along well-chosen axes was indeed a good approach for counting repetitions, even though it did not provide any details on implementation or even the approach to follow. Once the reps were extracted, we were able to proceed in the same way as the article by applying DTW matching to a reference video. However, the cosine similarity-based approach to scoring proved to be poor due to the noise generated by MediaPipe (which is not encountered in the article because the author uses the same camera angle as the reference video). As [4] seems to show, using angles between landmarks is more robust than directly using pose landmarks, so this is the method we finally favored, with a few adjustments.

3

PRACTICAL IMPLEMENTATION AND DETAILED APPROACH

3.1 DATASET CONSTRUCTION AND POSE ESTIMATION

In order to train our classification model and test our scoring functions, we needed to build a dataset of fitness videos. To do this, we started with a Kaggle dataset [3] of a wide variety of fitness exercise videos: filmed horizontally, vertically, with text sometimes covering the person, with cuts—which inevitably required cleaning. The dataset includes videos for 22 exercises, but to simplify the problem, we only considered 4: push-ups, squats, leg extension, and barbell bicep curls. The files for each selected exercise initially contained between 22 and 64 videos, and between 12,000 and 5,500 frames (Fig 1).

The data we worked with on this project are pose landmarks. To obtain them, we used the Python MediaPipe library, which detects key anatomical points on the following diagram: A first CNN model draws a box around the detected body to narrow down the search, determines the body’s orientation, and then determines the position of 33 key anatomical points (see Fig 2), which it can then diffuse from one image to another (the pose detection function is optimized so as not to calculate on each image). The output consists of the 3-dimensional coordinates of the 33 anatomical points: the points (x,y) correspond to the relative position according to the height and width of the video, while z is calculated relatively to the pelvis in an arbitrary unit—this is therefore pseudo-3D (only one camera), which is sufficient for our study but, as we will see in this report, may have contributed to model inaccuracies. We kept working with MediaPipe because the library is adapted for fast landmark calculations and runs very well on CPUs.

One initial difficulty encountered is the partial absence of landmarks on certain videos: for example, when Mediapipe is unsure of its detection (the probability of detecting the landmark is below an arbitrary threshold), the landmark is set to None. Furthermore, at certain moments in the videos, some parts of the body are hidden by an obstacle or even out of frame (see Fig 3). In order to have all 33 landmarks available on each video, we therefore created an interpolation function: when a landmark is set to None at time t , it is artificially recalculated linearly between $t_1 < t < t_2$, the times when the landmark is known closest to t , as follows: $x(t) = x_1 + (x_2 - x_1) \frac{t-t_1}{t_2-t_1}$. This allowed us to achieve greater robustness in our detections in order to obtain a complete dataset of landmarks. For each video in the dataset, we then collected the coordinates of the interpolated landmarks (MediaPipe parameters for detection and tracking confidence set to 0.3). Finally, to clean our dataset we established a scoring function for each sequence of landmarks, which averages out-of-bounds and unstable landmarks. For **unstability**, we calculated the discrete velocities of each landmark and we applied an RMS to increase the discrimination between stable and unstable sequences (often due to poor detection) before normalizing the obtained value. For each video, we then determined a final score calculated as follows: **final_score** = 0.3 **out_of_bonds** + 0.7 **unstability**, and we only kept videos with a score greater than 0.4 (an empirical value that satisfied us after viewing the

landmarks). After cleaning, we obtained a training dataset with 4,000 to 11,000 frames for each class.

3.2 POSE LANDMARKS NORMALIZATION

In order to facilitate video scoring and obtain consistent data for the classification model, we normalized the landmark coordinates. This is because the MediaPipe coordinates, which are relative to the video being played, are not suitable for a detailed study of the movements performed. To do so, we sought to create a regular database, invariant by translation and rotation (since the camera is not always in the same place), inspired by the same pre-processing as in the article [4]. Thus, for each set of landmarks, we defined the origin as the center of the hips, which resolves the translation invariance. For rotation invariance, we defined an orthonormal reference frame. First, we set the length of the torso (length between the origin and the center of the shoulders) as equal to 1, by dividing each landmark by this length. Then we constructed the x-axis as the right shoulder-left shoulder axis, the z-axis as the result of the vector product between x and the torso axis (center of the hips to the center of the shoulders), and finally we set the y-axis as the vector product between x and z, to ensure we had a good coordinate system. Nevertheless, after using vector products, the sign of each axis is rechecked so that y always points towards the head and z is oriented towards the person (looking at the sign of the shoulder-wrist vector). Once the new basis has been calculated, the pose landmarks are rotated to obtain a normalized version in an unified basis for the classification model.

However, this normalization has certain limitations: MediaPipe's calculation errors are directly reflected in the new calculated reference point, leading to fairly noisy coordinates from one exercise to another. To overcome this and facilitate classification, we also calculated additional features for each frame: different angles between triplets of landmarks (e.g., hip-knee-ankle) and distances between landmarks (head-shoulder, hip-ankle, etc.). We also experimented with other possible coordinate systems (e.g., first fixing the torso axis and calculating the other axes as before), but they proved to be less stable than the first one we chose.

3.3 EXERCISE CLASSIFICATION MODEL

After obtaining our first functional version of landmark extraction, we began working on exercise classification. The first question we asked ourselves was: should we classify entire videos or images one by one? Our limited data volume prompted us to explore the second approach first. The article [5] led us to the PointNet model [6]:

“Until recently, deep learning approaches for 3D data have been limited because of availability of techniques used to represent the data format. The PointNet model developed by Charles Qi and team from Stanford University in 2017 provides a deep learning framework capable of effectively handling 3D geometric data such as point clouds and meshes. The model is a unified architecture that directly takes point clouds as input and outputs either class labels for the entire input, or per point segment/part labels for each point of the input.

Though the model is frequently used for dense point cloud representations obtained from depth cameras or 3D scanners involving hundreds and thousands of points, it can also handle effectively classification tasks involving small point clouds such as the 3D landmark coordinates generated by the BlazePose model. The PointNet model is permutation invariant, which allows it to handle the unstructured nature of the point cloud data; transformation invariant, which enables it to process classification and segmentation tasks despite rotations and translation; and able to capture interactions and relative positions between the various points. This preserves its robustness in classification tasks when handling the 3D landmark outputs given that the orientation of the landmarks could be rotated or angled differently depending on the position of the camera and/or facing direction of the subject within the scene.”

Even though the PointNet model (Fig 5 describes its architecture) is not initially suited to our problem (our point cloud is not dense), we trusted the article and implemented this model. However, we encountered numerous difficulties. Many frames corresponding to the ends of repetitions are indistinguishable for different exercises. We initially worked with 22 different exercise classes, and the first attempts to train the model proved chaotic, with no improvement in accuracy metrics. To simplify the problem, we then restricted ourselves to 4 fairly distinct exercise classes. We also incorporated other adaptations, such as preventing images from the same video from being included in the training and evaluation datasets, and switching to the standardized version of landmark extraction that we had developed in the meantime. But once again, our attempts to train the model failed to get the learning curves off the ground.

Faced with these failures, we decided to try a video-based classification approach rather than an image-based one. This drastically reduced the amount of training data, prompting us to work with simpler models such as MLPs or CNN-1Ds. Two problems remained: each video is composed of a different number of frames, and the landmarks corresponding to a single video were a list of 5,000 to 50,000 items. For the first issue, we decided to sample our videos by keeping only 10 images, first randomly, then at regular intervals, as this gave better results. For the second issue, we chose to reduce the landmarks of each frame (given with the additional angles this time) to a lower dimension using PCA. Experiments showed that dimension 3 is the best compromise: beyond that, the model score does not increase. The choice to keep 10 images for sampling is arbitrary; we did not try larger values because the value 10 allowed us to achieve a very high score.

We reduced the input vectors to a size of 30. We then trained a fairly small MLP with a single hidden layer of size 100, which allowed us to exceed a score of 80% on our training dataset. We then moved on to a CNN-1D model, with two convolution layers that gradually reduce from 10 to 5 to 2 frames via max pooling, followed by two fully connected layers of size 128 that classify the exercise into one of 4 exercise classes. This model enabled us to achieve an accuracy of over 90%, and we kept it in our final pipeline. The learning curves are available in the appendix (Fig 6), as well as the distribution of classes in the training/test dataset (Fig 7).

Another optimization we tried to implement was to compromise between image and video classification by classifying video clips. Our initial approach was to try to divide the videos into repetitions of the exercise, but we encountered the following problem: to divide a video into repetitions, you need to know the nature of the exercise. The approaches we tried, such as searching for minima in a graph of the evolution of the PCA of landmarks, were not effective enough to overcome this problem. A second approach was to cut the videos into sequences of frames of an arbitrary number. Thus, to

classify a video, the model begins by cutting it into sequences and classifying each of these sequences, then keeps the majority vote. However, this approach did not work any better, and we assume that this is because it adds too much superfluous information to the model training. In particular, for some videos, there are several seconds between two repetitions, and the sequences corresponding to these two seconds are poor training data for the model. In other words, we would have had to completely rework the training dataset for this approach, so we decided to discard it because we were already satisfied with the first one. The t-SNE representations of the video data and sequence data are available in the appendix (Fig 8).

3.4 SYNCHRONIZATION AND GRADING METRIC

Once the class had been identified, we divided the video sequence into different repetitions (reps) and scored each one.

To detect reps, we used an approach based on the evolution of a given landmark for each exercise: specific landmarks can be identified as oscillating during the sequence of reps. For example, for push-ups, the landmark associated with the hand will oscillate along the z-axis, reaching its maximum when the arms are extended and its minimum when they are bent. Thus, in the `ExerciseClasses` class, we identified a landmark to study for each exercise, its axis of oscillation (x, y or z), and the extrema that defines the start of a rep (maximum or minimum) (see Fig 4). Once these characteristics have been established, we sought to identify the local extrema. In order to smooth out potential noise on the evolution of the landmark’s position, we applied a Savitzky-Golay filter, which acts as a low-pass filter: it attenuates abrupt variations while preserving the main features of the signal (amplitude, velocity, acceleration). We then identified the global minimum and maximum, which allowed us to calculate the maximum amplitude for the exercise. Then, using `scipy.signal.find_peaks`, we identified the local extrema of the signal, while ensuring that they are not too close to each other in time (we use the `min_dist` parameter of the function to do so) - for example, a push-up rep should not be done in less than a second (=30 frames) to be performed correctly. Once the extrema have been identified, only those that are close enough from the global extremum are kept, the notion of proximity being defined by a fixed ratio of the maximum amplitude. The fixed ratio is a percentage specific to each exercise, but on average equal to 35%. Once the starts of the reps were identified, we computed a table showing the index of the frames concerned (to which we add the first frame by default for the code to work properly).

The frame indices obtained allowed us to identify sequences of frames on which a single repetition is performed. In order to be able to note its correct execution, the idea we explored was to synchronize a reference rep (performed correctly by a professional—and with no ambiguity about the position of the landmarks) to the video so that we could then compare the movements performed frame by frame. To do this, we used a Dynamic Time Wrapping (DTW) method, as suggested in [1]. DTW is a technique that allows two time series of different lengths to be aligned by operating time dilations, compressions, and even pauses. For two sequences $s_1[n]$ and $s_2[n]$ of sizes N and M , DTW calculates an ascending path from $(0, 0)$ to (N, M) that minimizes the quantity $\sum_{(i,j) \in Path} \|s_1[i] - s_2[j]\|_2$, and returns it as a list of index pairs (see Fig 10). In our case, the two series being compared were the execution speeds of the reps (more robust than comparing positions), and the DTW path was then passed to the metric function for a frame-by-frame calculation.

An initial approach we implemented for the metric design was based on calculating the cosine similarity between the positions of landmarks on each frame of the two preceding sequences. For two vectors a and b , cosine similarity is defined as $\text{cossim}(a, b) = \frac{\langle a, b \rangle}{\|a\| \|b\|}$. For aligned vectors, this ratio is 1, and for orthogonal vectors, it is 0. However, inaccuracies in the position of the landmarks calculated with Mediapipe, or points that are not visible due to the camera angle have a significant impact on this metric, which did not meet the robustness criteria required for our study. Indeed, the slightest difference in normalization leads to a significative difference in the position of the entire skeleton, resulting in low scores on acceptable exercise performances. A second approach consisted of weighting the different landmarks according to their level of importance in the exercise observed (for example, we look more at the movement of the arms during push-ups than that of the head). But here again, the noise in the landmarks led to scores that did not meet our expectations. Finally, we opted to calculate the cosine similarity of a set of angles calculated from the landmarks (elbow, knee, foot angles, etc.). This approach, which is more robust than previous ones, made it possible to obtain similar scores for two equally good performances of an exercise taken from two different angles, which was the main criterion sought.

In order to refine our scoring system, we made some changes to our metrics: first, we weighted the angles according to the exercises so that we could better distinguish good and poor performances. Even though it helped to create a significative difference in the score between both types of performance, it was not enough for our study. We therefore chose to calculate the square of the weighted difference for each pair of angles, sum these squares in a negative exponential (which creates a good discrimination between poor and good reps), before readjusting everything using a non-centered sigmoid to obtain relevant scores between 0 and 1. For each exercise, the metric is adapted using different constants defined in `ExerciseClasses`, but the principle is the same.

Finally, a display function prints the video, the detected exercise class, counts the number of reps based on the number of frames already passed, and the average score for each rep using the metric defined previously (see Fig 11).

4

RESULTS AND CONCLUSION

At the end of this project, we succeeded in building a functional pipeline for classifying and scoring fitness exercises. Although the final result does not exactly match our initial expectations, we are generally very satisfied with the outcomes.

One satisfactory initial result concerns the classification of exercises. The CNN-1D model trained on compact representations of the videos (10 frames sampled and projected into 3 dimensions by PCA) achieved an accuracy of over 90% on our test set, with good separation between the four classes to which we restricted the project. The confusion matrix shows little systematic confusion, indicating that the model has successfully learned discriminating patterns despite the high variability of the training videos (viewing angle, quality, execution pace). This result is particularly encouraging given the small amount of data available to us. Repetition detection is also a strong point of the pipeline. Our approach, based on identifying characteristic oscillations of landmarks specific to each exercise,

has proven effective for real videos. Smoothing using a Savitzky-Golay filter combined with controlled selection of extrema allows for consistent repetition counting. Finally, the scoring of repetitions based on DTW synchronization and joint angle comparison yields qualitatively satisfactory results. The metric used makes it possible to distinguish correct movements, even when they are incomplete or poorly executed.

Despite these encouraging results, several limitations remain. The main one is the strong dependence on the quality of the landmarks extracted with MediaPipe: detection errors, body parts outside the field of view, or even inaccuracies on the z-axis calculated from a single 2D view are reflected at all stages of the pipeline, particularly during normalization and angle calculation. These errors are largely responsible for our difficulty in training PointNet and for the instability that can be observed in certain ratings. The number of exercises studied is also limited. Exercises with similar movements or static phases remain difficult to differentiate with current representations. Finally, rep scoring is based on numerous empirical parameters (landmark weights, thresholds, weightings, etc.) that are manually adjusted for each exercise. This approach, which is effective on a small scale, limits genericity and complicates its extension to new movements.

We would have liked to add a live pipeline to this project, which takes as input a live sports session filmed with a webcam rather than videos of each series of exercises. However, this would have raised many additional problems, such as learning to detect the wait between two sets of exercises, the problems inherent in real-time classification (you can't count repetitions until you've determined the exercise being performed), and, of course, the acceleration of computation time, which is not compatible with live use with a webcam in the current pipeline.

REFERENCES

- [1] P. Kapica, “Using pose estimation algorithms to build a simple gym training aid app,” *Medium*, 2019. [Online]. Available: <https://medium.com/@pawelkapica/using-pose-estimation-algorithms-to-build-a-simple-gym-training-aid-app-ef87b3d07f94>
- [2] J. Carreira, E. Noland, C. Hillier, and A. Zisserman, “A Short Note on the Kinetics-700 Human Action Dataset,” arXiv preprint, 2023.
- [3] Philosopher0808, *Gym Workout Exercises Video Dataset*, Kaggle, 2021. [Online]. Available: <https://www.kaggle.com/datasets/philosopher0808/gym-workoutexercises-video>
- [4] R. Vemulapalli, F. Arrate, and R. Chellappa, “Human action recognition by representing 3D skeletons as points in a Lie group,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014, pp. 588–595.
- [5] Wesley, “3D Human Pose Classification using MediaPipe and PointNet,” *Medium*, 2023. [Online]. Available: <https://medium.com/@wesley/3d-human-pose-classification-using-mediapipe-and-pointnet>
- [6] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, “PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 652–660. [Online]. Available: <https://arxiv.org/pdf/1612.00593>
- [7] H. Kotte, M. Kravčík and N. Duong-Trung, “Real-Time Posture Correction in Gym Exercises: A Computer Vision-Based Approach for Performance Analysis, Error Classification and Feedback”, in *CEUR Workshop Proceedings*, vol. 3499, 2023. [Online]. Available: <https://ceur-ws.org/Vol-3499/paper9.pdf>

ANNEX & FIGURES

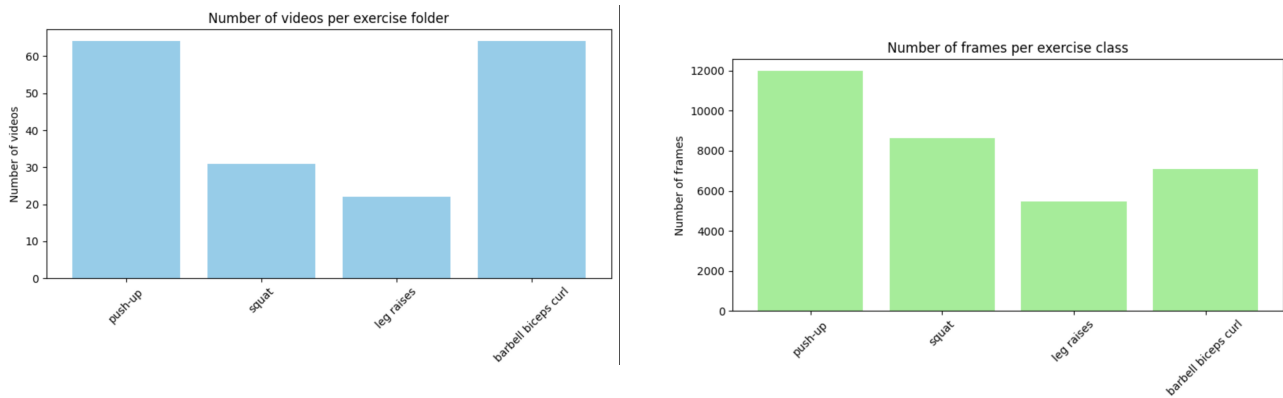


Figure 1: Distribution of the number of videos and frames regarding the exercise

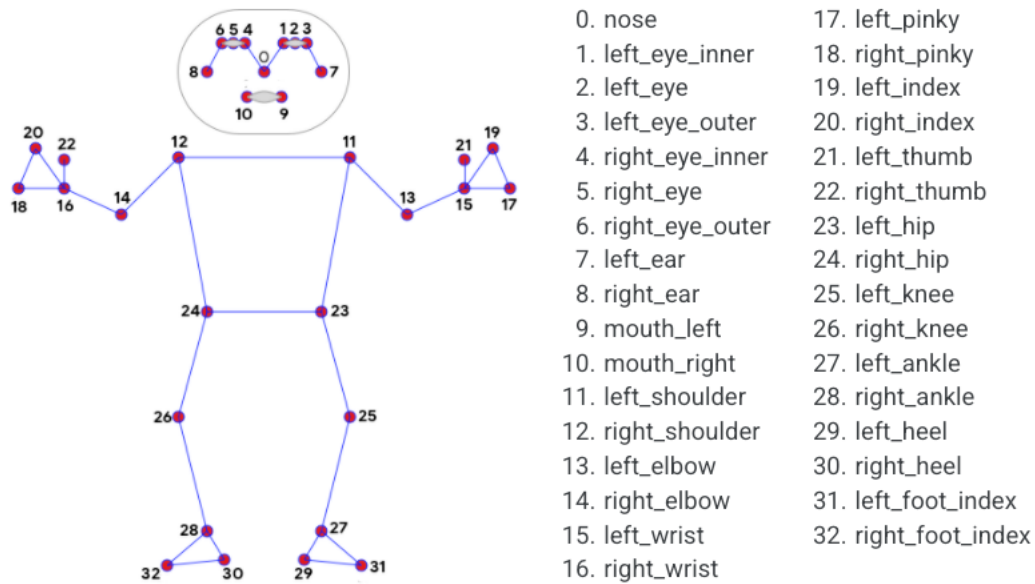
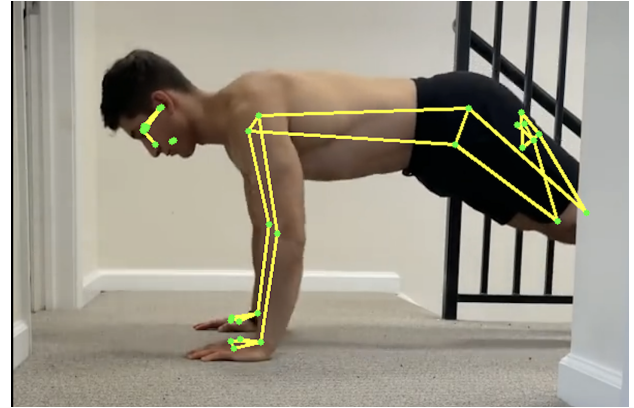


Figure 2: 33 landmarks identified by MediaPipe and their labels



(a) Some missing MediaPipe landmarks



(b) Interpolation may magnify some errors

Figure 3: Some landmarks extraction errors due to MediaPipe



Figure 4: Variations of marker 15 (left hand) along the z-axis during a series of push-ups

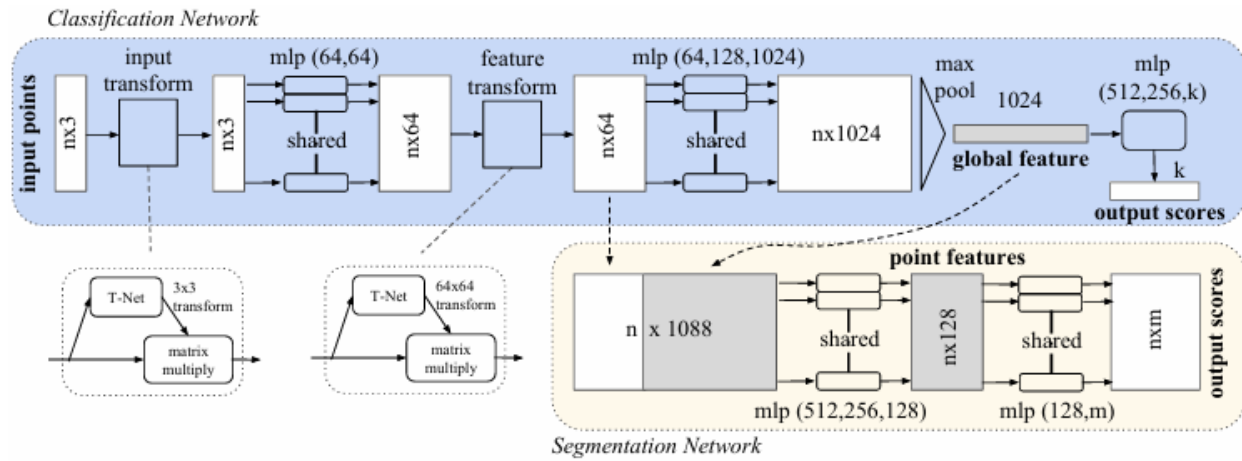


Figure 5: Architecture of Pointnet model

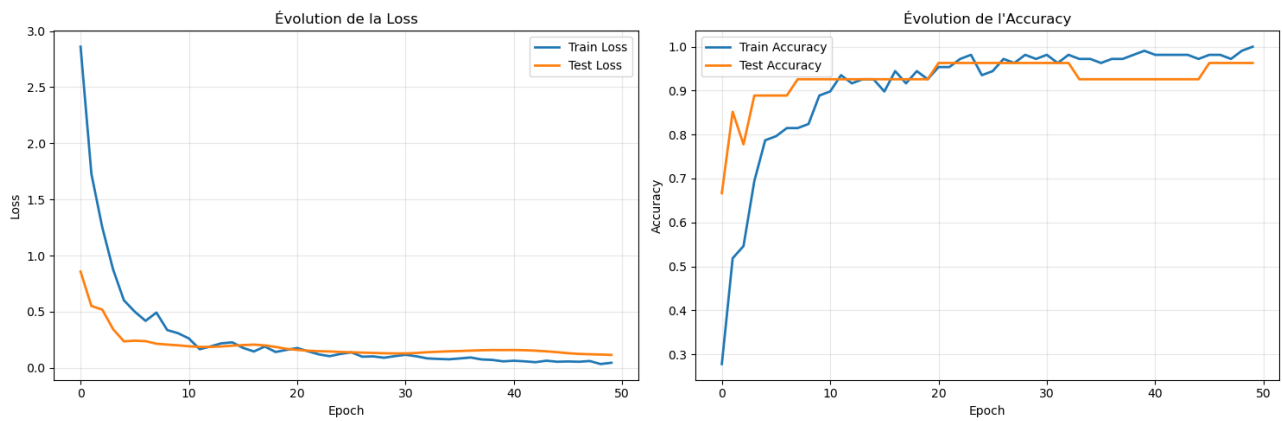


Figure 6: Learning curves of the final CNN-1D model

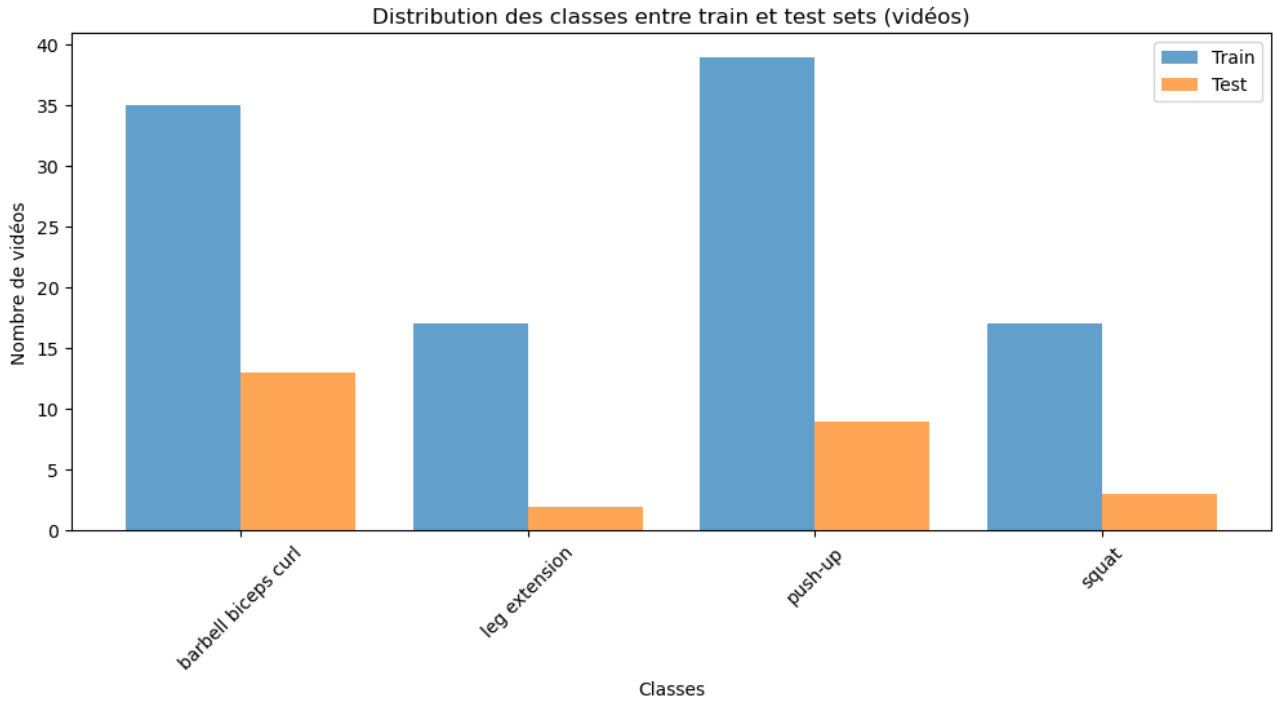


Figure 7: Repartition of the classes of the videos between the train and the test set

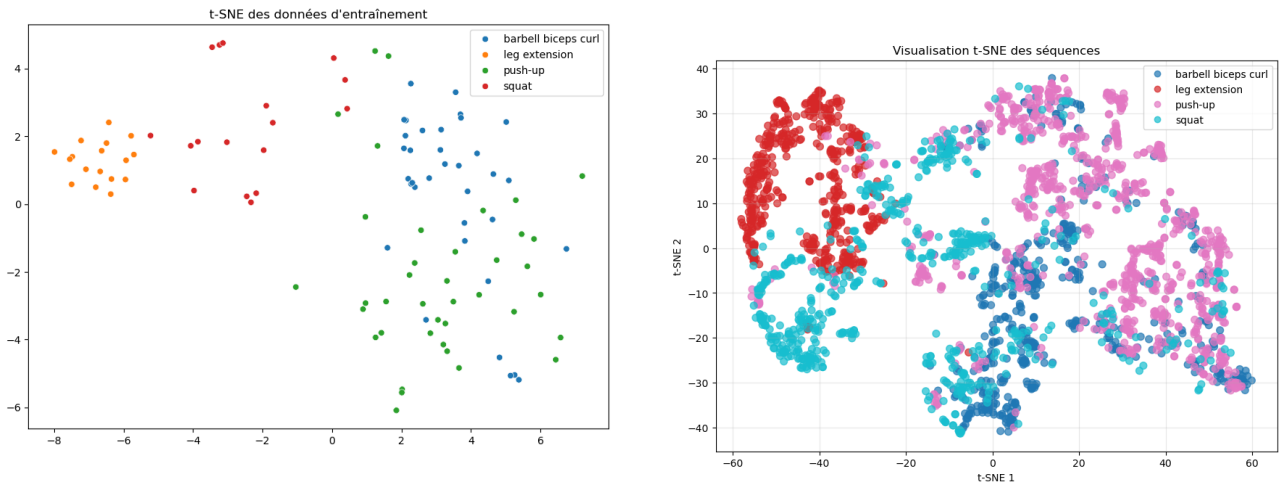


Figure 8: t-SNE representation of the dataset, for video data and sequence data

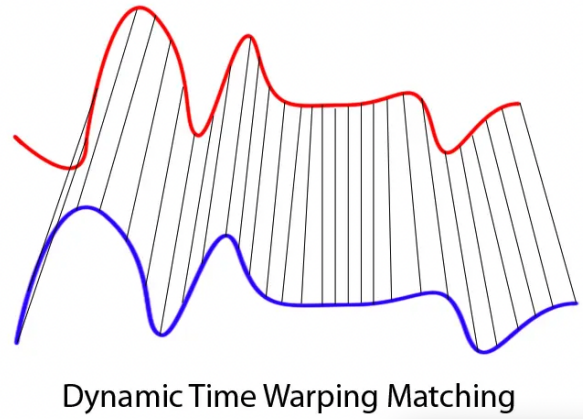
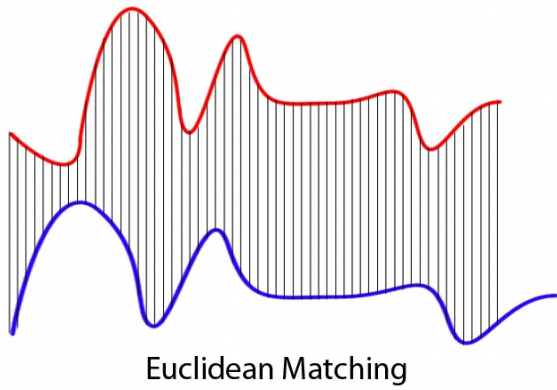
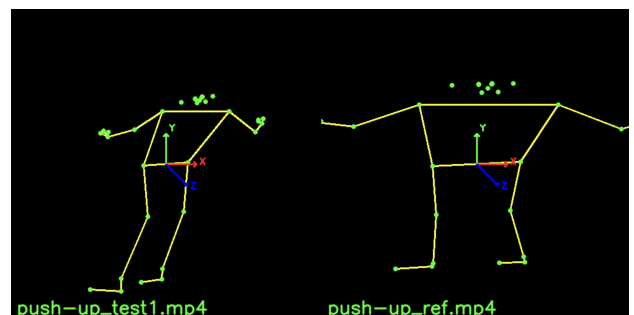


Figure 9: Comparison between classic Euclidean matching and Dynamic Time Warping.
Source: Wikimedia.org



(a) Video point of view



(b) Skeleton point of view

Figure 10: Reference and test videos synchronized using DTW

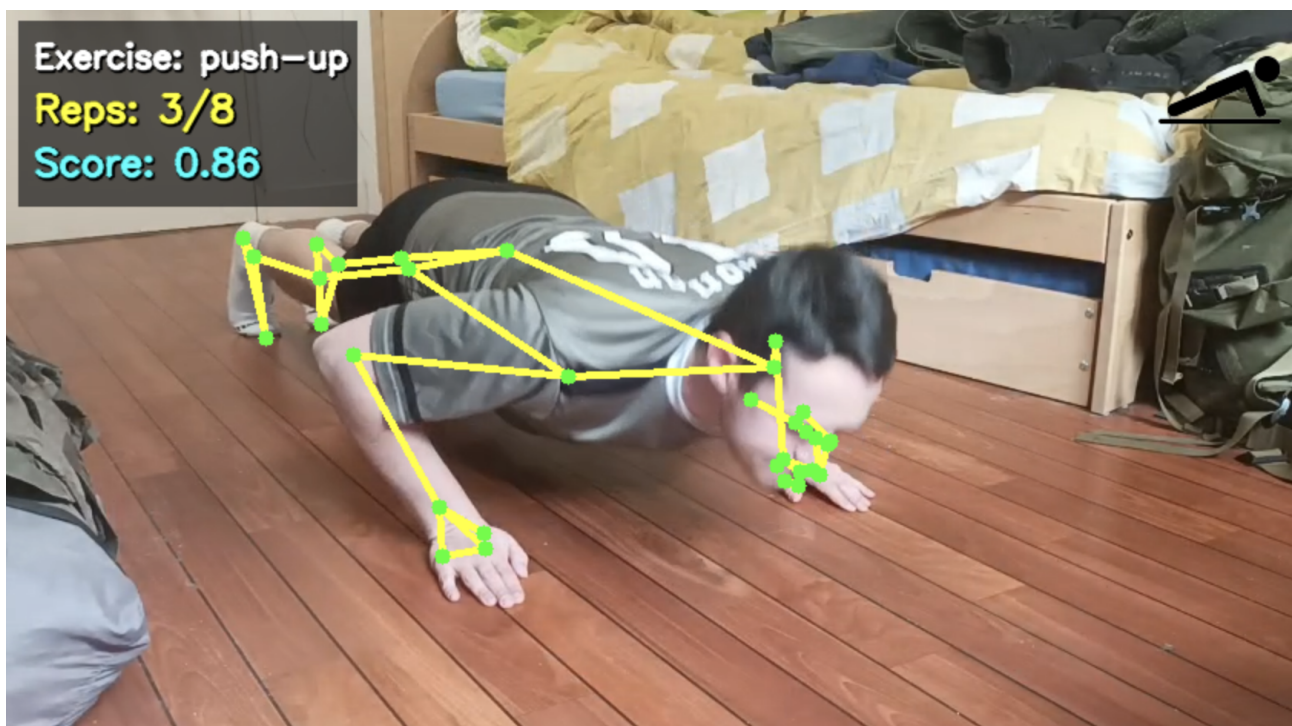


Figure 11: Final display of our program : exercise predicted, number of reps and score